CAREER DEVELOPMENT PLAYBOOK

Senior → Principal Engineer

A 12-month, project-led roadmap for engineering excellence, technical leadership, cross-team influence, and durable organizational impact.

70% applied work | 20% feedback & mentoring | 10% study

Prepared for Iryna Tyshchenko
Azure, distributed systems, and agentic-systems emphasis
Version 1.0 • 24 October 2026

How to use this roadmap

This is not a reading list or a title checklist. Every phase produces an observable artifact, an operational behavior, and evidence that another person can evaluate. Run it against real work whenever possible. If your current role does not expose you to the required scope, create a bounded internal proposal, open-source implementation, or production-style capstone.

> **Operating rule:** do not advance because you "covered the topics." Advance when the completion criteria are demonstrably true.

Level transition

Dimension	Senior	Staff	Principal
Scope	Complex work within one team	Ambiguous initiatives spanning teams	Technical direction across a domain or organization
Time horizon	Current quarter	Multiple quarters	12–24 months and strategic options
Leverage	Own delivery and mentor peers	Align teams and create reusable approaches	Change how the organization makes technical decisions
Evidence	Reliable delivery and operations	Cross-team outcomes and adoption	Durable, measurable impact that continues without you

Public engineering frameworks consistently separate levels by increasing scope, influence, ambiguity, and impact rather than by coding volume alone.[1][2][28] Promotion evidence should show sustained performance at the target level and impact, not simply time served.[3]

Table of contents

1. Baseline and evidence system
2. Engineering fundamentals
3. Continuous delivery
4. System-design judgment
5. Reliability and operations
6. Security engineering
7. Business and cost fluency
8. Technical communication
9. Influence without authority
10. Mentoring and delegation
11. Cross-team initiative
12. Technical strategy
13. Azure and agentic specialization
14. Organizational mechanisms
15. Promotion portfolio
16. 12-month execution calendar

Section sources

1. [1] GitLab Engineering Career Framework
2. [2] GitLab Principal Engineer Framework
3. [3] Dropbox Promotion Principles
4. [28] Dropbox Engineering Career Framework

1 Baseline and evidence system

Week 1

Establish the gap between your present behavior and the target level. Use evidence, not confidence, as the unit of progress.

Actions

1. Score yourself from 1–5 across coding, architecture, delivery, operations, security, business judgment, communication, influence, mentoring, and strategy.
2. Ask your manager and two experienced partners to score the same dimensions using specific examples.
3. Select one high-value initiative that can expand from team scope to multi-team scope over the year.
4. Create an evidence log with columns for date, problem, action, decision, collaborators, result, metric, and supporting link.
5. Define three missing behaviors—not vague traits—that your roadmap must develop.

Artifacts

- ☐ One-page development charter
- ☐ 10-dimension baseline scorecard
- ☐ Named target initiative and sponsor
- ☐ Evidence log with monthly review cadence

Completion criterion: your manager can explain what target-level evidence would look like and agrees that the selected initiative can produce it.

Failure mode: turning the roadmap into private study. Senior-to-principal growth requires externally visible outcomes and feedback.

Section sources

1. [1] GitLab — Engineering Career Framework
2. [3] Dropbox — Promotion Principles

2 Engineering fundamentals and repository discipline

Months 1–2

Build a workflow in which safe changes are easy and risky changes are visible. GitHub Flow uses branches and pull requests around a deployable default branch, while protected branches can require reviews and status checks.[4][5] Google's review guidance emphasizes design, functionality, complexity, tests, naming, comments, style, documentation, and context.[6]

Practices to establish

Area	Expected practice	Evidence
Branching	Trunk-based workflow; short-lived branches; no long-running environment branches	Median branch age and merge frequency
Pull requests	Small, single-purpose, linked to an outcome; review checklist; required checks	PR size, review latency, revert rate
Commits	Atomic changes, clear messages, reversible steps	Readable history and easier rollback
Quality	Formatter, linter, type checks, static analysis, dependency scans	Automated checks on every change
Testing	Unit logic, integration boundaries, contracts, a small critical E2E suite	Fast feedback and tracked flaky tests

Project

Improve one existing repository: protect the default branch, add a pull-request template, enforce checks, document the branching policy, quarantine or fix flaky tests, and provide a one-command local verification path.

> **Completion criterion:** a new engineer can clone, test, change, review, and safely merge the project using the documented path.

Section sources

1. [4] GitHub Docs — GitHub Flow
2. [5] GitHub Docs — Protected Branches
3. [6] Google — What to Look for in a Code Review

3 Continuous delivery and measurable flow

Months 2–3

Own work from problem definition through production learning. DORA describes continuous delivery as keeping software deployable and using automation, testing, and small batches; its delivery metrics balance throughput and instability rather than optimizing speed alone.[7][8][9]

Capabilities

1. Turn requests into measurable outcomes and explicit acceptance criteria.
2. Split large changes into independently releasable increments.
3. Automate build, test, security scanning, packaging, and deployment.
4. Use feature flags and backward-compatible migrations for risky changes.
5. Define rollback and roll-forward paths before launch.
6. Instrument the outcome, not only the deployment.

Track as a balanced set

Change lead time Commit to production	**Deployment frequency** Useful release cadence	**Failed deployment recovery** Time to restore after failure
Change fail rate Deployments requiring remediation	**Deployment rework** Unplanned corrective work	**Outcome metric** Customer or business effect

Project

Lead one production feature end-to-end. Publish the delivery plan, dependency map, risk register, release/rollback procedure, and a two-week outcome review.

> **Completion criterion:** the release is repeatable, observable, recoverable, and evaluated against its intended outcome.

Section sources

1. [7] DORA — Continuous Delivery
2. [8] DORA — Test Automation
3. [9] DORA — Software Delivery Metrics

4 System-design judgment

Months 3–4

Move beyond pattern recall. Good system design starts from workload requirements and explicitly balances reliability, security, cost, operational excellence, and performance efficiency.[10][11]

Design loop

1. State functional requirements, quality attributes, constraints, scale, and non-goals.
2. Model normal flow and failure flow.
3. Compare at least two viable options.
4. Quantify capacity, latency, availability, recovery, and cost assumptions.
5. Address data ownership, consistency, idempotency, retries, timeouts, versioning, and migrations.
6. Record the decision, its consequences, and the conditions that would invalidate it.

Required artifact: design proposal

Include context, system boundaries, architecture diagram, APIs/events, data model, threat model, observability, capacity assumptions, failure modes, rollout, rollback, operational ownership, cost estimate, alternatives, and success metrics.

Practice scenarios

- High-volume order ingestion with replay and duplicate suppression
- Multi-tenant agent platform with per-tenant policy boundaries
- Event-driven customer-support workflow with human approval
- Migration from synchronous orchestration to asynchronous processing

> **Completion criterion:** reviewers can disagree with your decision while still understanding the requirements, evidence, assumptions, and trade-offs that produced it.

Section sources

1. [10] Microsoft — Azure Application Design Principles
2. [11] Microsoft — Azure Well-Architected Framework

5 Reliability, observability, and incident learning

Months 4–5

Reliability is a product decision. Google SRE recommends defining service level objectives from user-relevant indicators, and using error budgets to balance release velocity with reliability work.[12][13] Blameless postmortems focus on systemic learning and tracked corrective action.[14]

Build the operating model

1. Select user-visible service-level indicators for availability, latency, correctness, or freshness.
2. Define SLOs and an error-budget policy with product stakeholders.
3. Add structured logs, metrics, traces, correlation IDs, and business events.
4. Create actionable alerts tied to user impact; remove noisy alerts.
5. Write runbooks for detection, mitigation, recovery, and escalation.
6. Test backups and recovery rather than assuming they work.
7. Run a failure exercise and facilitate a blameless review.

Project

Adopt SLOs for one service and remove one recurring failure class. Measure pages, recovery time, error-budget consumption, and repeat incidents before and after.

> **Completion criterion:** the team knows when reliability is acceptable, when releases should slow, and how to restore service under pressure.

Section sources

1. [12] Google SRE — Service Level Objectives
2. [13] Google SRE — Error Budget Policy
3. [14] Google SRE — Postmortem Culture

6 Security engineering and secure delivery

Month 5

NIST's Secure Software Development Framework organizes practices for preparing the organization, protecting software, producing well-secured software, and responding to vulnerabilities.[15] Threat modeling should identify assets, trust boundaries, threats, mitigations, and residual risk before implementation hardens the wrong design.[16]

Core practices

- Use least privilege, managed identity, secret stores, and short-lived credentials.
- Separate authentication, authorization, and business-policy decisions.
- Validate external input and encode output at trust boundaries.
- Scan source, dependencies, containers, infrastructure, and leaked secrets in CI.
- Maintain dependency ownership and a vulnerability-response process.
- Define data classification, retention, encryption, deletion, and audit requirements.
- Abuse-case test the highest-risk user and agent actions.

Project

Threat-model the roadmap's target system. Resolve the highest-risk design issue, add a security regression test, and document residual risk with an accountable owner.

> **Completion criterion:** security controls are testable parts of delivery—not a final review meeting.

Section sources

1. [15] NIST — Secure Software Development Framework
2. [16] OWASP — Threat Modelling Control

7 Business, customer, and cost fluency

Month 6

Technical leadership requires economic judgment. FinOps frames cloud value as a shared responsibility across engineering, finance, and business roles and emphasizes informed, timely decisions rather than cost cutting in isolation.[17]

Decision questions

1. Who experiences the problem, and how often?
2. Which customer, revenue, risk, compliance, or productivity metric changes?
3. What is the cost of delay?
4. What is the smallest investment that can test the assumption?
5. What are build, operate, migrate, and opportunity costs?
6. What evidence would make us stop or change direction?

Technical-debt portfolio

Classify debt by customer impact, delivery drag, operational risk, security exposure, and reversibility. Prioritize the debt that blocks business options; do not defend rewrites with aesthetic arguments.

Project

Write a one-page investment memo comparing three options: maintain, incrementally improve, or replace. Include baseline, expected benefit, effort range, risk, opportunity cost, and a measurable decision checkpoint.

> **Completion criterion:** a non-engineering leader can understand why the investment matters, what uncertainty remains, and how success will be measured.

Section sources

1. [17] Microsoft — FinOps Framework Overview
2. [11] Microsoft — Azure Well-Architected Framework

8 Technical communication and decision records

Months 6–7

Architecture decision records preserve context, alternatives, decisions, and consequences so future teams do not have to reconstruct reasoning.[18] Clear technical writing reduces coordination cost; Google's technical-writing curriculum emphasizes clarity, audience, structure, and concise language.[19]

Four formats to master

Format	Lead with	Must include
Design document	Problem and recommendation	Constraints, options, trade-offs, risks, rollout
Executive update	Decision/status and impact	Evidence, risk, required action
ADR	Decision	Context, alternatives, consequences
Incident update	Customer impact	Current state, mitigation, next update

Practice

1. Write the one-sentence conclusion first.
2. Separate facts, assumptions, recommendations, and decisions.
3. Use diagrams to clarify boundaries and flow—not to decorate.
4. Tailor depth to the audience while preserving the same underlying facts.
5. Close every decision with owner, date, next action, and review condition.

> **Completion criterion:** stakeholders can repeat the decision, rationale, owner, risk, and next step without another meeting.

Section sources

1. [18] Google Cloud — Architecture Decision Records
2. [19] Google — Technical Writing Courses

9 Influence without authority

Month 7

Cross-team decisions need explicit roles and decision rules. GitLab's public process distinguishes accountable decision-makers, consulted participants, and contributors to avoid consensus-by-default and unclear ownership.[20]

Stakeholder map

Question	Capture
Who decides?	One accountable decision owner
Who is affected?	Users, operators, security, finance, dependent teams
What do they optimize?	Speed, reliability, autonomy, cost, compliance, roadmap
What could block agreement?	Conflicting incentives, hidden dependencies, migration cost
How will conflict resolve?	Decision rule, escalation route, deadline

Exact structure

> "We need to decide **X** because it affects **Y**. The viable options are **A/B/C**. I recommend **B** because of these constraints. The main risk is **Z**. I need a decision from **owner** by **date**."

Project

Align at least two teams on a shared technical decision. Pre-wire concerns through individual conversations, publish the options, facilitate the review, record dissent, and assign migration ownership.

> **Completion criterion:** agreement survives beyond the meeting because the decision, responsibilities, and exception path are explicit.

Section sources

1. [20] GitLab — Decision-Making Process
2. [18] Google Cloud — Architecture Decision Records

10 Mentoring, delegation, and engineering leverage

Months 7–8

High-level engineers improve the quality and autonomy of others. Google's engineering-practices materials make review standards explicit and reusable, turning individual judgment into a shared mechanism.[6][21]

Delegation contract

- **Outcome:** what changes for users or the system?
- **Boundaries:** what is fixed, negotiable, or out of scope?
- **Authority:** which decisions can the owner make independently?
- **Checkpoints:** where is feedback cheap?
- **Escalation:** which risks require immediate discussion?
- **Evidence:** how will completion be evaluated?

Mentoring progression

1. Demonstrate the method.
2. Pair while the learner drives.
3. Review artifacts at checkpoints.
4. Transfer ownership and stakeholder visibility.
5. Remain available without reclaiming the work.

Project

Delegate a meaningful component or workstream to another engineer. Give them design and stakeholder visibility, not just implementation tasks. Record where your coaching increased independent decision quality.

> **Completion criterion:** another engineer can lead the area without routing routine decisions through you.

> **Failure mode:** becoming the expert bottleneck while calling it "high standards." Principal-level leverage reduces dependency on the principal.

Section sources

1. [21] Google — Engineering Practices Documentation
2. [6] Google — Code Review Guidance

11 Lead a cross-team technical initiative

Months 8–10

Choose an organizational problem that cannot be solved by writing one service. DORA's platform-engineering guidance emphasizes internal platforms as products that reduce cognitive load and enable developer self-service rather than imposing another ticket queue.[22]

Initiative sequence

1. Establish customer and engineering baselines.
2. Name the executive or domain sponsor and accountable technical owner.
3. Map participating teams, dependencies, incentives, and migration costs.
4. Define outcomes and non-goals.
5. Compare options and record the decision.
6. Pilot with one willing team.
7. Measure adoption, friction, reliability, and time saved.
8. Iterate before broad rollout.
9. Create self-service documentation, support, and exception paths.
10. Transfer permanent ownership and retire obsolete paths.

Good initiative candidates

- Shared deployment or observability capability
- API/event standardization and compatibility tooling
- Secure agent-tool gateway and evaluation framework
- Cloud-cost visibility and ownership model
- Service template with production-readiness defaults

> **Completion criterion:** at least two teams adopt the capability, measurable friction falls, and continued use does not depend on your manual intervention.

Section sources

1. [22] DORA — Platform Engineering
2. [9] DORA — Software Delivery Metrics

12 Technical strategy and portfolio direction

Months 10–11

Strategy connects business change to a coherent sequence of technical capabilities and deliberate trade-offs. Microsoft's Cloud Adoption Framework organizes work around strategy, planning, readiness, adoption, governance, security, and management rather than treating migration as a purely technical event.[23]

Strategy structure

1. **Diagnosis:** current system, business forces, constraints, and evidence.
2. **Principles:** decision rules that resolve recurring trade-offs.
3. **Capabilities:** what must become possible.
4. **Portfolio:** invest, maintain, migrate, consolidate, or retire.
5. **Sequence:** dependencies and three horizons—now, next, later.
6. **Governance:** owners, measures, review cadence, and exception process.
7. **Options:** what is intentionally deferred and what would trigger reconsideration.

Required artifact

Produce a 12–18-month strategy for one domain, with baseline, target capabilities, architecture direction, investment themes, deprecation plan, organizational dependencies, risk, cost range, and outcome measures.

> **Completion criterion:** teams can use the strategy to make consistent local decisions without asking you to decide every detail.

Section sources

1. [23] Microsoft — Cloud Adoption Framework
2. [11] Microsoft — Azure Well-Architected Framework

13 Deep specialization: Azure and agentic systems

Throughout

Develop recognized depth without losing broad engineering judgment. Microsoft Foundry treats generative-AI observability as monitoring quality, safety, performance, and operational behavior through traces and evaluation.[24][25] OWASP warns that excessive agency arises when models receive unnecessary functionality, permissions, or autonomy; minimize extensions and privileges and require authorization for consequential actions.[26]

Depth map

Area	Demonstrable capability
Azure architecture	Identity, networking, compute, data, messaging, observability, resilience, cost
Agent design	Bounded tool contracts, state, planning, fallback, human approval
Evaluation	Task datasets, deterministic checks, rubric/judge evaluation, regression gates
Observability	Trace prompts, models, tools, retrieval, latency, tokens, cost, decisions
Security	Prompt-injection defenses, least privilege, data boundaries, audit, safe failure

Capstone

Build a production-style customer-support or commerce agent with authenticated tool access, least-privilege permissions, human approval for consequential writes, traceable decisions, evaluation datasets, quality/latency/cost gates, CI/CD, threat model, failure injection, runbook, and an architecture decision log.

> **Completion criterion:** the system can demonstrate not only a successful demo, but also how it fails, how risk is bounded, how quality is measured, and how an operator investigates behavior.

Section sources

1. [24] Microsoft Foundry — Generative AI Observability
2. [25] Microsoft Foundry — Evaluate Generative AI Applications
3. [26] OWASP — Excessive Agency

14 Organizational mechanisms and paved roads

Month 11

A mechanism turns expertise into a repeatable organizational capability. Backstage software templates illustrate a self-service approach in which teams create components from standardized skeletons and published workflows.[27]

Candidate mechanisms

- Production-readiness checklist and automated evidence
- Secure service or agent template
- Standard CI/CD and policy-as-code
- Architecture-review and ADR templates
- SLO onboarding and observability defaults
- Agent evaluation harness and risk controls
- Dependency and vulnerability ownership dashboard

Mechanism quality test

1. Does it solve a repeated, expensive problem?
2. Is the happy path self-service?
3. Are defaults safer than custom work?
4. Can teams extend or exit when necessary?
5. Is adoption and value measured?
6. Is there a permanent owner and review cadence?

> **Completion criterion:** multiple teams use the mechanism voluntarily or through an agreed standard, and measurable effort, risk, or variance declines.

Section sources

1. [27] Backstage — Software Templates
2. [22] DORA — Platform Engineering

15 Promotion portfolio and readiness review

Month 12

Package evidence around target-level behavior and organizational outcomes. Dropbox's promotion principles emphasize demonstrated next-level impact rather than promotion as a reward for tenure or effort alone.[3] Public career frameworks help calibrate scope, but your employer's framework remains authoritative.[1][28]

Evidence story for each initiative

1. Problem and baseline
2. Your specific role and decisions
3. Scope, ambiguity, and stakeholders
4. Alternatives and trade-offs
5. Obstacles and conflict handled
6. Measured customer, business, reliability, delivery, or cost result
7. Reusable mechanism or leaders developed
8. What continued working without you

Portfolio table

Competency	Evidence expected	Example proof
Architecture	Multi-system trade-off and durable decision	Design + ADR + measured result
Delivery	Predictable incremental execution	Milestones + DORA/outcome metrics
Influence	Alignment across conflicting incentives	Stakeholder feedback + adopted decision
Leverage	Others and systems scale your impact	Adoption + delegated ownership
Strategy	Direction guides multiple teams	Roadmap decisions linked to strategy

> **Completion criterion:** reviewers can independently trace each claim to an artifact, metric, or collaborator—and see sustained target-level scope rather than one exceptional project.

Section sources

1. [3] Dropbox — Promotion Principles
2. [1] GitLab — Engineering Career Framework
3. [28] Dropbox — Engineering Career Framework

12-month execution calendar

Q1 — Foundation
Baseline
Repository discipline
Continuous delivery

Q2 — Production judgment
System design
Reliability
Security
Business fluency

Q3 — Leverage
Communication
Influence
Mentoring
Cross-team pilot

Q4 — Principal scope
Cross-team rollout
Strategy
Mechanism
Promotion portfolio

Month	Primary outcome	Required artifact
1	Calibrated target and engineering baseline	Development charter + repository assessment
2	Safe, fast change workflow	Protected flow + automated checks
3	End-to-end delivery ownership	Released feature + outcome review
4	Explicit architecture judgment	Reviewed design + ADRs
5	Reliable and secure operations	SLOs + threat model + recovery test
6	Business-grounded investment decisions	Technical investment memo
7	Clear decisions and stakeholder alignment	Decision record + cross-team agreement
8	Expanded ownership through others	Delegated workstream + mentoring evidence
9	Cross-team pilot	Baseline + pilot metrics
10	Measured adoption	Rollout report + ownership transfer
11	Domain direction and reusable mechanism	Strategy + paved road
12	Verified target-level portfolio	Evidence packet + feedback

Weekly operating rhythm

1. **Monday:** name one outcome that matters more than task volume.
2. **Tuesday:** reserve two hours for architecture or technical depth.
3. **Wednesday:** mentor, delegate, or remove a blocker for another engineer.
4. **Thursday:** advance one decision or stakeholder alignment.
5. **Friday:** record evidence, metrics, risk, and lessons.
6. **Monthly:** request specific feedback from a manager and two collaborators.
7. **Quarterly:** re-score the baseline and replace weak activities with stronger evidence opportunities.

Final readiness checklist

Senior Engineer

- □ Leads a complex team-level project from ambiguity to production outcomes
- □ Makes explicit architecture trade-offs and records consequential decisions
- □ Operates services with tested reliability, security, and recovery practices
- □ Improves engineering workflow rather than only completing assigned tasks
- □ Mentors peers and communicates effectively with product and operations

Principal Engineer

- □ Leads initiatives across multiple teams and conflicting incentives
- □ Sets technical direction for a domain over a 12–24 month horizon
- □ Connects architecture to customer value, cost, risk, and organizational capacity
- □ Creates mechanisms adopted beyond the originating team
- □ Develops other technical leaders and transfers meaningful ownership
- □ Demonstrates measurable, durable impact that continues without constant involvement
- □ Preserves options and reduces systemic risk under uncertainty

> **Calibration note:** titles differ by company. Use this roadmap to build behaviors and evidence, then map them to your employer's official career framework and promotion process.

Quarterly review questions

1. Which outcome changed because of my technical leadership?
2. Where did I improve another team's ability to act independently?
3. Which decision did I make clearer, safer, or more reversible?
4. What evidence shows adoption or durable impact?
5. Where am I still acting as a bottleneck?

References

1. GitLab. *Engineering Career Framework.* https://handbook.gitlab.com/handbook/engineering/careers/matrix/
2. GitLab. *Engineering Career Framework: Principal.* https://handbook.gitlab.com/handbook/engineering/careers/matrix/principal/
3. Dropbox. *Promotion Principles.* https://dropbox.github.io/dbx-career-framework/promotion_principles.html
4. GitHub Docs. *GitHub Flow.* https://docs.github.com/en/get-started/using-github/github-flow
5. GitHub Docs. *About Protected Branches.* Protected branches documentation
6. Google. *What to Look for in a Code Review.* https://google.github.io/eng-practices/review/reviewer/looking-for.html
7. DORA. *Continuous Delivery.* https://dora.dev/capabilities/continuous-delivery/
8. DORA. *Test Automation.* https://dora.dev/capabilities/test-automation/
9. DORA. *Software Delivery Metrics.* https://dora.dev/guides/dora-metrics/
10. Microsoft. *Azure Application Design Principles.* Microsoft Learn
11. Microsoft. *Azure Well-Architected Framework.* Microsoft Learn
12. Google SRE. *Service Level Objectives.* https://sre.google/sre-book/service-level-objectives/
13. Google SRE. *Error Budget Policy.* https://sre.google/workbook/error-budget-policy/
14. Google SRE. *Postmortem Culture.* https://sre.google/workbook/postmortem-culture/
15. NIST. *Secure Software Development Framework.* https://csrc.nist.gov/projects/ssdf
16. OWASP. *Threat Modelling Control.* OWASP DSOVS
17. Microsoft. *FinOps Framework Overview.* Microsoft Learn
18. Google Cloud. *Architecture Decision Records.* Google Cloud Architecture Center
19. Google. *Technical Writing Courses.* https://developers.google.com/tech-writing
20. GitLab. *Decision-Making Process.* GitLab Handbook
21. Google. *Engineering Practices Documentation.* https://google.github.io/eng-practices/
22. DORA. *Platform Engineering.* https://dora.dev/capabilities/platform-engineering/
23. Microsoft. *Cloud Adoption Framework.* Microsoft Learn
24. Microsoft Foundry. *Observability in Generative AI.* Microsoft Learn
25. Microsoft Foundry. *Evaluate Generative AI Applications.* Microsoft Learn
26. OWASP GenAI Security Project. *Excessive Agency.* OWASP GenAI
27. Backstage. *Software Templates.* https://backstage.io/docs/features/software-templates/
28. Dropbox. *Engineering Career Framework.* https://dropbox.github.io/dbx-career-framework/

Sources were accessed on 24 October 2026. This roadmap synthesizes public guidance; it does not replace an employer's internal leveling framework or promotion policy.